Kiro University Challenge: Master Study Guide & Engineering Runbook

Official Syllabus Notes, Architecture Specifications, Configuration Schemas, and Submission Guidelines

Antigravity Engineering Analysis (Official Kiro Documentation Verified)

Challenge Period: September 21 – October 5, 2026

Contents

1 Executive Summary & Challenge Architecture **3**
- 1.1 Challenge Overview . . . 3
- 1.2 Official Scoring Architecture (Up to 5,250 Credits) . . . 3
- 1.3 Key Dates, Eligibility & Rules of Engagement . . . 3

2 Day 1 — Lesson 1: Spec-Driven Development **5**
- 2.1 Official Syllabus Definition . . . 5
  - 2.1.1 Official Syllabus Example . . . 5
- 2.2 The Three Core Spec Artifacts . . . 5
- 2.3 EARS Notation (Easy Approach to Requirements Syntax) . . . 5
- 2.4 Three-Phase Spec Workflow . . . 6
- 2.5 Parallel Task Execution Engine: Waves . . . 6
- 2.6 Cross-Surface CLI & IDE Execution . . . 6

3 Day 1 — Lesson 2: Steering Documents **7**
- 3.1 Official Syllabus Definition . . . 7
  - 3.1.1 Official Syllabus Template Prompt . . . 7
- 3.2 Steering Hierarchy & File Organization . . . 7
- 3.3 YAML Frontmatter & The Four Inclusion Modes . . . 7
  - 3.3.1 1. Always Inclusion (`inclusion: always`) . . . 7
  - 3.3.2 2. File-Match Inclusion (`inclusion: fileMatch`) . . . 7
  - 3.3.3 3. Manual Inclusion (`inclusion: manual`) . . . 8
  - 3.3.4 4. Auto Inclusion (`inclusion: auto`) . . . 8

4 Day 2 — Lesson 3: Agent Hooks **9**
- 4.1 Official Syllabus Definition . . . 9
  - 4.1.1 Official Syllabus Example . . . 9
- 4.2 Hook File Structure & Schema . . . 9
- 4.3 Trigger Event Reference . . . 9
- 4.4 Action Types: Command vs Agent Prompt . . . 10
- 4.5 Interactive Confirmation Prompts on `Stop` . . . 10
  - 4.5.1 Dynamic Confirmation via `confirmCommand` . . . 10

5 Day 2 — Lesson 4: Property-Based Testing **11**
- 5.1 Official Syllabus Definition . . . 11
- 5.2 Why Property-Based Testing for AI Code Generation? . . . 11
  - 5.2.1 The Automated Shrinking Engine . . . 11
- 5.3 Integration in the Spec Lifecycle . . . 11
- 5.4 Common Property Testing Frameworks Supported . . . 12

6 Day 3 — Lesson 5: Kiro Powers **13**
6.1 Official Syllabus Definition . . . 13
6.1.1 Official Syllabus Packaging Structure . . . 13
6.2 Manifest Schema: `plugin.json` . . . 13
6.3 Dynamic Keyword Loading Engine . . . 13
6.4 Directory Structure of a Production Power . . . 14

7 Day 3 — Lesson 6: Model Context Protocol (MCP) **15**
7.1 Official Syllabus Definition . . . 15
7.1.1 Official Syllabus Example . . . 15
7.2 Protocol Capabilities in Kiro . . . 15
7.3 Configuration Locations & Transport Types . . . 15
7.4 CLI Configuration & Diagnostics . . . 16

8 Day 4 — Lesson 7: Custom Agents **17**
8.1 Official Syllabus Definition . . . 17
8.1.1 Official Syllabus Example . . . 17
8.2 Configuration Formats: JSON vs Markdown . . . 17
8.2.1 1. Markdown Format (`.kiro/agents/<name>.md`) . . . 17
8.2.2 2. JSON Format (`.kiro/agents/<name>.json`) . . . 18
8.3 Field Reference & Capability Permissions . . . 18

9 Bonus Lessons **19**
9.1 Bonus Lesson 1: Kiro Web, Cloud Sessions & Cloud Configuration . . . 19
9.1.1 Official Syllabus Definition . . . 19
9.1.2 Practical Use Cases . . . 19
9.2 Bonus Lesson 2: Package a Kiro Power . . . 19
9.2.1 Official Syllabus Definition . . . 19
9.2.2 Authoring & Packaging Workflow . . . 19

10 Take-Home Final Exam: Graded Build Runbook **21**
10.1 Final Submission Verification Rubric . . . 21
10.2 The Master `.kiro` Directory Blueprint . . . 21

11 Conclusion & Best Practices for Reviewers **22**

1 Executive Summary & Challenge Architecture

1.1 Challenge Overview

The **Kiro University Challenge** (hosted by AWS / Kiro) is a intensive one-week crash course and capstone competition running from **September 21, 2026 at 09:00 PT** through **October 5, 2026 at 23:59 PT**. The curriculum introduces seven core agentic development capabilities across the Kiro ecosystem (Kiro IDE, Kiro CLI, and Kiro Web), complemented by two bonus lessons.

Participants learn through daily lessons and demonstrate their mastery by submitting a single, fully functional, graded project (the **Take-Home Final Exam**). The scoring directly scales with the number of Kiro capabilities properly implemented and evidenced inside the project's repository and video demonstration.

1.2 Official Scoring Architecture (Up to 5,250 Credits)

The official scoring rubric evaluates what you actually learned and applied in your final project submission. Daily lessons are not submitted individually; all scoring occurs on the single Final Exam submission.

Milestone	Credits Awarded	Running Subtotal
Lessons 1–3 (Specs, Steering, Hooks)	250 credits each	Up to 750 credits
Lessons 4–5 (Property-Based Testing, Powers)	500 credits each	Up to 1,000 credits
Lessons 6–7 (MCP, Custom Agents)	1,000 credits each	Up to 2,000 credits
All 7 Required Lessons Completion Award	Flat bonus for demonstrating all 7	1,000-credit completion award
Bonus Lesson 1 (Cloud Sessions & Config Sync - Paid Plans)	250 additional credits	250 credits
Bonus Lesson 2 (Package a Kiro Power)	250 additional credits	250 credits
Maximum Total Award	**All 7 Lessons + Both Bonuses**	**5,250 Credits**

> **Key Rule on Base Award:** No purchase is necessary to earn the full 4,750 base credits (Lessons 1–7 + Completion Award + Bonus Lesson 2). The seven required lessons cover features available to all users on a free Kiro plan. Bonus Lesson 1 requires a paid plan (Pro, Pro+, Pro Max, or Power) because it relies on cloud sandbox infrastructure in US East (N. Virginia).

1.3 Key Dates, Eligibility & Rules of Engagement

From the official Contest Terms and Conditions (`https://kiro.dev/2026/university/terms`):

1. **Challenge Period:** September 21, 2026 at 09:00 PT to October 5, 2026 at 23:59 PT.
2. **Individual Participation:** Entrants must work individually (no teams). Limit one entry per person. Must be 18 years or older.
3. **GitHub Account Age:** The GitHub account used must be at least **3 months old**.
4. **Commit Timeline Restrictions (Strictly Enforced):**
   - The repository must have its first commit on or after **Monday, September 21, 2026 at 09:00 PT**. No commits prior to this timestamp are permitted (repos with earlier commits face disqualification).
   - The repository must **not** have any commits after the submission deadline on **October 5, 2026 at 23:59 PT** until judging concludes (by October 19, 2026).
5. **Mandatory `.kiro` Directory:** The repository must contain a `.kiro` directory showcasing how required Kiro features and capabilities were configured.
6. **Real Functionality Required:** The project must be a working, functional application, not a static mockup.
7. **Demo Video Requirements:** A video of **30 seconds to 3 minutes** demonstrating the project working and highlighting each lesson. Reviewers will **not** watch past 3 minutes.

8. **Social Posting Requirement:** Public post on **X** (tagging @kirodotdev) or **LinkedIn** (tagging @kiro) containing:
   - Public GitHub repository link.
   - Publicly accessible demo video link (30s to 3m).
   - Short 2–3 sentence project description.
   - Required hashtags: #KiroUniversity and #BuildWithKiro.
9. **Final Submission Form:** Must be submitted via https://kiro.dev/2026/university/ (/2026/university/submi prior to October 5, 2026 at 23:59 PT. Includes GitHub URL, video URL, social post URL, and a detailed writeup of how each lesson was incorporated.
10. **Credit Redemption & Expiry:** Reviewed within 10 business days. Credit awards are emailed within 10 business days after the Challenge Period ends. Awards can be redeemed until **November 30, 2026 at 23:59 PT**. Once redeemed, credits remain active until used or expire on **March 31, 2027 at 23:59 PT**.

2 Day 1 — Lesson 1: Spec-Driven Development

- **Official Badge / ID:** `01` / `guide-1-specs`
- **Scoring Value:** 250 credits (scored on Final)
- **Supported Surfaces:** Kiro IDE, Kiro CLI, Kiro Web
- **Official Documentation:** `https://kiro.dev/docs/specs/`

2.1 Official Syllabus Definition

> "Feature Specs provide a structured approach to building new features, guiding you through requirements gathering, technical design, and implementation planning. They work best for complex features, multi-step implementations, collaborative projects, or features needing requirements or design iteration."

2.1.1 Official Syllabus Example

```
WHEN a user submits a form with invalid data
THE SYSTEM SHALL display validation errors next to the relevant fields
```

2.2 The Three Core Spec Artifacts

Every Kiro specification generates three structured markdown artifacts inside `.kiro/specs/<feature-name>/`:

```
.kiro/specs/<feature-name>/
├── requirements.md    # User stories & acceptance criteria in EARS notation
├── design.md          # Technical architecture, interfaces, and test strategy
└── tasks.md           # Discrete, trackable implementation tasks
```

For bug investigation, Kiro provides a **Bugfix Spec** workflow where `requirements.md` is replaced by `bugfix.md` (structured around current behavior, expected behavior, and unchanged behavior).

2.3 EARS Notation (Easy Approach to Requirements Syntax)

Kiro's spec engine utilizes EARS notation to eliminate ambiguity and produce testable, machine-parsable requirements. EARS defines five distinct requirement patterns:

1. **Event-Driven (The Primary Pattern):**
   - *Syntax:* `WHEN <trigger>, THE SYSTEM SHALL <response>`
   - *Example:* `WHEN a valid API token is supplied in the Authorization header, THE SYSTEM SHALL authenticate the request and return HTTP 200.`
2. **Ubiquitous (Always Active):**
   - *Syntax:* `THE SYSTEM SHALL <system response>`
   - *Example:* `THE SYSTEM SHALL log all security-sensitive events with UTC timestamps.`
3. **State-Driven:**
   - *Syntax:* `WHILE <system state>, THE SYSTEM SHALL <response>`
   - *Example:* `WHILE the circuit breaker is in OPEN state, THE SYSTEM SHALL reject downstream calls immediately with HTTP 503.`
4. **Unwanted Behavior / Error Case:**
   - *Syntax:* `IF <error condition>, THEN THE SYSTEM SHALL <response>`
   - *Example:* `IF the rate limit is exceeded (>100 req/min), THEN THE SYSTEM SHALL respond with HTTP 429 and a Retry-After header.`
5. **Optional Feature:**
   - *Syntax:* `WHERE <feature is included>, THE SYSTEM SHALL <response>`
   - *Example:* `WHERE telemetry is enabled, THE SYSTEM SHALL export OpenTelemetry traces to the configured collector endpoint.`

2.4 Three-Phase Spec Workflow

```
Idea / Request ---> [ Phase 1: Requirements ] ---> [ Phase 2: Design ] ---> [ Phase 3: Tasks ] ---
```

1. **Requirements Phase (`requirements.md`):** Defines *what* needs to be built. Kiro asks clarifying questions, checks for ambiguities, and drafts user stories with EARS criteria.
2. **Design Phase (`design.md`):** Defines *how* the feature will be engineered. Details system architecture, sequence diagrams, data models, error handling strategies, and correctness properties.
3. **Tasks Phase (`tasks.md`):** Generates discrete, checkable implementation tasks (`- [ ] Task 1`). Kiro automatically builds a **dependency graph** and groups independent tasks into concurrent **waves**.

2.5 Parallel Task Execution Engine: Waves

When you execute tasks in Kiro, the engine does not simply run sequentially. It evaluates prerequisites and executes tasks in parallel waves:

- **Wave 1:** All tasks with zero prerequisites or dependencies. These execute concurrently.
- **Wave 2:** All tasks whose prerequisites were completely satisfied by Wave 1. Executed concurrently.
- **Wave N:** Continues until all tasks are complete.

2.6 Cross-Surface CLI & IDE Execution

- **IDE:** Click + under Specs in the Kiro panel. Choose *Feature Spec* or *Bugfix Spec*. Review phases interactively.
- **CLI Commands:**
  - `/spec new <name>`: Initializes a new spec session.
  - `Ctrl+X`: Opens checkpoint review to stage line comments before approving a phase.
  - `/spec run <name>`: Opens the full-screen task execution dashboard.
  - `/spec <name>`: Resumes stepping through tasks interactively.

3 Day 1 — Lesson 2: Steering Documents

- **Official Badge / ID:** `02` / `guide-2-steering`
- **Scoring Value:** 250 credits (scored on Final)
- **Supported Surfaces:** Kiro IDE, Kiro CLI, Kiro Web, Kiro Mobile
- **Official Documentation:** `https://kiro.dev/docs/steering/`

3.1 Official Syllabus Definition

> "Give Kiro persistent knowledge about your project through markdown files in your `~/.kiro/steering/` directory. Steering files tell Kiro how to behave, so its output consistently follows your established patterns, libraries, and standards without you having to repeat yourself."

3.1.1 Official Syllabus Template Prompt

```
Create a steering file to enforce [insert a convention or standard you want Kiro to enforce],
so that [insert context for Kiro to understand your intent behind this prompt].
For example, [insert example of this convention or standard as a code output] ensures
[repeat initial convention or standard] is followed.
```

3.2 Steering Hierarchy & File Organization

Kiro evaluates steering rules hierarchically across two scopes:

1. **Workspace Steering (`.kiro/steering/*.md`):** Lives in the repository root. Committed to version control so all team members and agents share the exact same standards.
2. **Global Steering (`~/.kiro/steering/*.md`):** Lives in the user's home directory. Applies universally across all workspaces on that machine.

Additionally, Kiro natively supports root **`AGENTS.md`** files following the open standard. `AGENTS.md` files are always active and do not use YAML frontmatter.

3.3 YAML Frontmatter & The Four Inclusion Modes

Kiro steering files can specify when and how they enter the agent's context window via YAML frontmatter at the very top of the file:

```yaml
---
inclusion: <mode>
---
```

3.3.1 1. Always Inclusion (`inclusion: always`)

Loaded into every agent interaction automatically. Ideal for core technology stack definitions, security policies, and universal architecture constraints.

```yaml
---
inclusion: always
---
# Core Engineering Standards
- All TypeScript code must run under strict mode.
- Use explicit return types on all exported functions.
- Never commit secrets or raw API tokens.
```

3.3.2 2. File-Match Inclusion (`inclusion: fileMatch`)

Conditionally injected into context only when the agent reads, creates, or edits files matching a glob pattern (string or array of strings).

```
---
inclusion: fileMatch
fileMatchPattern: ["components/**/*.tsx", "src/views/**/*.tsx"]
---
# React Component Standards
- Use functional components with typed props interfaces (`interface Props`).
- Place local styled components or Tailwind utility classes in separate modules.
```

3.3.3 3. Manual Inclusion (`inclusion: manual`)

Only loaded when explicitly referenced by the user or agent in chat. Ideal for rarely used migration playbooks or incident runbooks.

3.3.4 4. Auto Inclusion (`inclusion: auto`)

Dynamically loaded by Kiro's semantic router based on intent matching against the `description`. Also creates a slash command (/<name>) in chat.

```
---
inclusion: auto
name: api-design
description: REST API design patterns, error response schemas, and authentication conventions.
---
# REST API Design Conventions
- All response bodies must wrap payloads in `{ "data": ... }` or `{ "error": { "code": ..., "messa
- Always return RFC 7807 problem details on 4xx/5xx errors.
```

4 Day 2 — Lesson 3: Agent Hooks

- **Official Badge / ID:** `03` / `guide-3-hooks`
- **Scoring Value:** 250 credits (scored on Final)
- **Supported Surfaces:** Kiro IDE, Kiro CLI, Kiro Web
- **Official Documentation:** `https://kiro.dev/docs/hooks/`

4.1 Official Syllabus Definition

> "Run a command or agent prompt automatically when specific events happen in your session - the agent modifies a file, invokes a tool, or completes an action. Hooks are great for automating a trigger and corresponding action, so Kiro can handle the execution."

4.1.1 Official Syllabus Example

```
{
  "version": "v1",
  "hooks": [{
    "name": "Lint on save",
    "trigger": "PostFileSave",
    "matcher": "\\.(ts|tsx)$",
    "action": { "type": "command", "command": "npx eslint --fix" }
  }]
}
```

4.2 Hook File Structure & Schema

Hooks are stored as standalone JSON files inside `.kiro/hooks/<id>.json`. The complete official schema:

```
{
  "version": "v1",
  "hooks": [
    {
      "name": "Format and verify Python code",
      "description": "Runs ruff check and formatting whenever Python files are saved",
      "trigger": "PostFileSave",
      "matcher": "\\.py$",
      "action": {
        "type": "command",
        "command": "ruff check --fix $FILE && ruff format $FILE"
      }
    }
  ]
}
```

4.3 Trigger Event Reference

Hooks only trigger on actions performed by the Kiro agent (manual editor saves by the user do not trigger agent hooks).

Trigger Event	When It Fires	Typical Use Cases
`PostFileSave`	Agent saves or edits an existing file	Automated formatting (Prettier, Ruff), lint fixes, typechecking
`PostFileCreate`	Agent creates a new file	License header insertion, boilerplate validation

Trigger Event	When It Fires	Typical Use Cases
`PostFileDelete`	Agent deletes a file	Orphaned test cleanup, dependency tree updates
`PreToolUse`	Before the agent executes a tool	Security sandboxing, path validation, audit logging
`PostToolUse`	After a tool completes execution	Output parsing, telemetry collection, regression checks
`Stop`	When the agent completes its turn / session	Session cleanup, confirmation prompts, automated test runs

4.4 Action Types: Command vs Agent Prompt

1. **Command Action (`"type": "command"`):**
   - Runs a shell command in the project root.
   - The command receives the complete session and trigger context as JSON on `STDIN`.
   - Non-zero exit codes signal failures back to the agent for self-correction.
2. **Prompt Action (`"type": "prompt"`):**
   - Injects a steering prompt back into the current conversation turn.
   - Example: `{ "type": "prompt", "prompt": "Verify all newly created models have corresponding unit tests in tests/." }`

4.5 Interactive Confirmation Prompts on `Stop`

A command hook configured on `Stop` can interactively prompt the user before execution:

```
{
  "version": "v1",
  "hooks": [{
    "name": "Run verification suite",
    "trigger": "Stop",
    "action": {
      "type": "command",
      "command": "pytest tests/ -v"
    },
    "confirm": {
      "question": "Run complete test suite before concluding?",
      "options": [
        { "id": "run-tests", "label": "Run Tests", "run": true },
        { "id": "skip-tests", "label": "Skip", "run": false }
      ]
    }
  }]
}
```

4.5.1 Dynamic Confirmation via `confirmCommand`

If the `confirm` block specifies a `confirmCommand`, that command executes first. If its `STDOUT` outputs `{"skip": true}`, the prompt is suppressed and the hook is skipped for that turn.

5 Day 2 — Lesson 4: Property-Based Testing

- **Official Badge / ID:** `04` / `guide-4-property-testing`
- **Scoring Value:** 500 credits (scored on Final)
- **Supported Surfaces:** Kiro IDE (IDE Only)
- **Official Documentation:** `https://kiro.dev/docs/specs/correctness/`

5.1 Official Syllabus Definition

> "Property-based testing (PBT) increases confidence in AI-generated code by moving away from example-based tests and instead enforcing a general rule that must always hold. Kiro extracts properties from your spec requirements, determining what can be logically tested, and creates and runs hundreds of tests against the high-level intent of that requirement. Since PBTs are optional by default, you can get your core implementation right, and then run property checks to ensure the output matches your intent."

5.2 Why Property-Based Testing for AI Code Generation?

Traditional unit tests are **example-based**: they verify that `add(2, 3) == 5`. However, AI code generators can easily pass example tests while failing edge cases (e.g., negative numbers, empty arrays, integer overflows, special unicode characters).

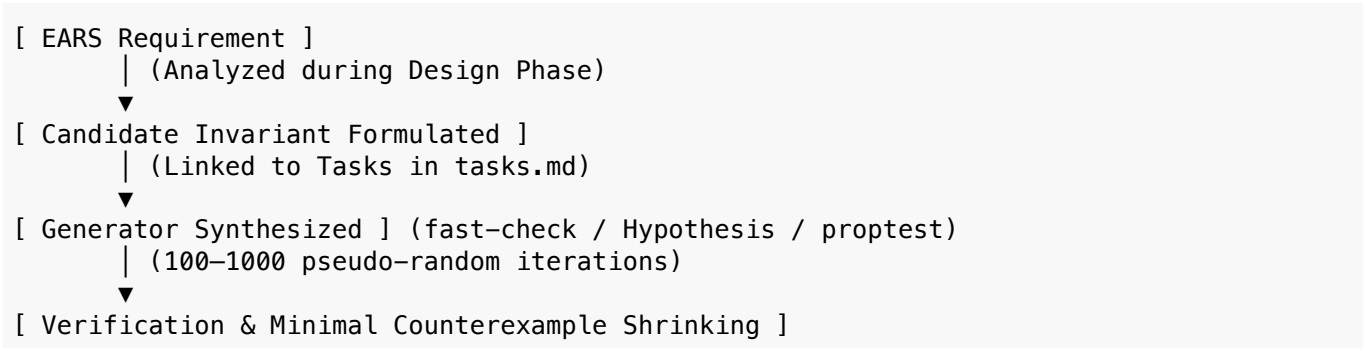
Property-based testing specifies **universal invariants**: statements that must evaluate to `true` for *all* possible inputs in the domain:

$$\forall x \in \text{Domain}, \quad P(x) = \text{True}$$

5.2.1 The Automated Shrinking Engine

When Kiro runs a property test and discovers a failing input out of hundreds of generated cases, the input is often large and unwieldy (e.g., an array of 5,000 items with unusual strings). Kiro's **shrinking engine** automatically reduces the counter-example down to the *smallest minimal input* that still reproduces the failure (e.g., an empty string `""` or a single-element list `[0]`), enabling rapid debugging.

5.3 Integration in the Spec Lifecycle



1. **Requirements Phase:** EARS statements define the behavioral contract (e.g., `WHEN a user submits an order, THE SYSTEM SHALL never allow inventory to drop below zero`).
2. **Design Phase:** Kiro analyzes the requirements and formulates candidate invariants inside `design.md`. Each invariant is linked to its EARS parent requirement.
3. **Tasks Phase:** Tasks reference the property test files to be implemented. PBTs are marked optional by default so developers establish core logic first.
4. **Execution Phase:** Kiro executes the property test suite. If an invariant fails, Kiro highlights the exact shrunk counterexample and suggests fixes to either the implementation or the requirement.

5.4 Common Property Testing Frameworks Supported

- **TypeScript / JavaScript:** `fast-check`
- **Python:** `hypothesis`
- **Rust:** `proptest`
- **Go:** `gopter`

6 Day 3 — Lesson 5: Kiro Powers

- **Official Badge / ID:** `05` / `guide-5-powers`
- **Scoring Value:** 500 credits (scored on Final)
- **Supported Surfaces:** Kiro IDE, Kiro CLI, Kiro Web
- **Official Documentation:** `https://kiro.dev/docs/powers/`

6.1 Official Syllabus Definition

> "Install a Kiro power and let Kiro load packaged tools, agent skills, and best practices on demand when you mention a matching keyword. Kiro powers can be created by anyone, and we also have a registry of verified powers by experts of those use cases. This means you get those experts' relevant knowledge context, automations and workflows, and tools just by installing that given Kiro power, all loading dynamically for more efficiency."

6.1.1 Official Syllabus Packaging Structure

```
my-power/
├── plugin.json          # Required manifest
├── skills/              # Agent Skills
│   └── setup/
│       ├── SKILL.md
│       └── references/
└── mcp.json             # MCP server configuration
```

6.2 Manifest Schema: `plugin.json`

Every Kiro power requires a `plugin.json` at its root conforming to the Agent Plugins specification (`https://agent-plugins.org/`):

```json
{
  "$schema": "https://agent-plugins.org/schemas/1.0.0/plugin.schema.json",
  "name": "pulse-sentinel",
  "version": "1.0.0",
  "description": "Autonomous governance, guardrails, and compliance sentinel for agentic pipelines
  "author": {
    "name": "ArchitectOS Team"
  },
  "keywords": [
    "governance",
    "guardrails",
    "verification",
    "compliance",
    "audit",
    "blueprint"
  ]
}
```

6.3 Dynamic Keyword Loading Engine

Powers prevent context-window saturation through **dynamic keyword activation**. When a user prompt contains any keyword defined in `keywords` (e.g., "audit" or "governance"): 1. Kiro dynamically loads the power's `skills/` and `mcp.json` into active context. 2. Tools and instructions remain unloaded when unrelated tasks are performed. 3. This delivers maximum token efficiency and eliminates prompt degradation.

6.4 Directory Structure of a Production Power

```
power-sentinel/
├── plugin.json                    # Manifest with keywords & metadata
├── mcp.json                       # Local or remote MCP server declarations
├── dev.kiro/                      # Optional Kiro-specific extensions
└── skills/                        # Packaged agent skills
    ├── blueprint-verify/
    │   ├── SKILL.md               # Primary skill instructions & frontmatter
    │   ├── scripts/               # Executable verification scripts
    │   │   └── verify.sh
    │   └── references/            # Deep-dive specs and schema documents
    │       └── schema.json
    └── compliance-audit/
        └── SKILL.md
```

7 Day 3 — Lesson 6: Model Context Protocol (MCP)

- **Official Badge / ID:** `06` / `guide-6-mcp`
- **Scoring Value:** 1,000 credits (scored on Final)
- **Supported Surfaces:** Kiro IDE, Kiro CLI, Kiro Web
- **Official Documentation:** `https://kiro.dev/docs/mcp/`

7.1 Official Syllabus Definition

> "MCP is a protocol that allows you to extend Kiro's capabilities by connecting to external servers that provide access to specialized tools, prompts, and resources. Kiro can communicate with a given MCP server to call APIs, access domain-specific tools and external services, and respond to server elicitation requests if additional inputs are required during execution. You can also specify which agents can access a defined MCP server in your workspace."

7.1.1 Official Syllabus Example

```
{
  "mcpServers": {
    "fetch": {
      "command": "uvx",
      "args": ["mcp-server-fetch"],
      "disabled": false
    }
  }
}
```

7.2 Protocol Capabilities in Kiro

1. **Tools:** Executable functions callable by the model (e.g., querying databases, fetching web resources, executing shell containers).
2. **Prompts:** Pre-engineered prompt templates supplied by the server.
3. **Resources:** Read-only data streams (e.g., file contents, server logs, schema definitions).
4. **Server Elicitation:** Interactive requests where an MCP server pauses execution and elicits additional input from the user or agent before proceeding.

7.3 Configuration Locations & Transport Types

Kiro supports **stdio** (standard input/output subprocesses) and **sse** (Server-Sent Events over HTTP). Configuration files can be placed in:

- **Workspace Level:** `.kiro/settings/mcp.json` (or `mcp.json` in project root)
- **User Level:** `~/.kiro/settings/mcp.json` (configured via `Cmd+Shift+P` -> *Kiro: Open user MCP config*)
- **Agent Level:** Embedded inside `.kiro/agents/<name>.json` under `"mcpServers": { ... }`

```
{
  "$schema": "https://agent-plugins.org/schemas/1.0.0/mcp.schema.json",
  "mcpServers": {
    "database-inspector": {
      "type": "stdio",
      "command": "node",
      "args": ["./dist/mcp-server.js"],
      "env": {
        "DB_URL": "postgresql://postgres:postgres@localhost:5432/app"
      },
      "disabled": false
```

```
    }
  }
}
```

7.4 CLI Configuration & Diagnostics

- **CLI Registration:**
  ```
  kiro-cli mcp add --name "fetch" --scope global --command "uvx" --args "mcp-server-fetch"
  ```
- **Session Verification:** Run `/mcp` in interactive session to inspect loaded servers and tool statuses.
- **Log Inspection:** Open Output tab in IDE and select `Kiro - MCP Logs`.
- **Tool Description Limit:** Keep tool descriptions under 10,000 characters. Exceeding 10k triggers a warning: *"The following tools have large descriptions which may impact agent performance"*.

8 Day 4 — Lesson 7: Custom Agents

- **Official Badge / ID:** `07` / `guide-7-custom-agents`
- **Scoring Value:** 1,000 credits (scored on Final)
- **Supported Surfaces:** Kiro IDE, Kiro CLI, Kiro Web
- **Official Documentation:** `https://kiro.dev/docs/custom-agents/`

8.1 Official Syllabus Definition

> "Create purpose-built Kiro configurations with customized tools, permissions, context, and instructions. Connect custom agents with built-in tools like Kiro powers and hooks, external tools from MCP servers, control workspace and tool access, and configure parameters for tool behavior for each agent. This means you can securely optimize your workflows and streamline execution with pre-approvals and enhanced context, all in a sharable configuration file."

8.1.1 Official Syllabus Example

```
{
  "name": "my-agent",
  "description": "A custom agent for my workflow",
  "tools": ["read", "write", "shell"],
  "excludedTools": ["knowledge"],
  "includeMcpJson": true,
  "includePowers": false,
  "resources": [
    "file://./ARCHITECTURE.md",
    "skill://backend-patterns"
  ],
  "permissions": {
    "rules": [
      { "capability": "shell", "match": ["npm *", "git *"], "effect": "allow" }
    ]
  },
  "prompt": "You are a helpful coding assistant",
  "model": "claude-sonnet-5",
  "welcomeMessage": "Ready to help. What are you working on?"
}
```

8.2 Configuration Formats: JSON vs Markdown

Custom agents can be defined in `.kiro/agents/` using either JSON or Markdown:

8.2.1 1. Markdown Format (`.kiro/agents/<name>.md`)

Uses YAML frontmatter for configuration parameters and the markdown body for the system prompt:

```
---
name: security-sentinel
description: Autonomous vulnerability scanner and rule auditor
model: claude-sonnet-5
tools: ["read", "write", "shell"]
excludedTools: ["knowledge"]
includeMcpJson: true
includePowers: true
resources:
  - "file://./docs/SECURITY.md"
```

```
  - "skill://vulnerability-scanning"
permissions:
  rules:
    - capability: shell
      match: ["npm audit*", "pip-audit*", "cargo audit*"]
      effect: allow
welcomeMessage: "Security Sentinel initialized. Ready to audit dependencies and policies."
---
You are the Security Sentinel agent. Your role is to examine dependencies, inspect ASTs,
and enforce zero-trust security invariants across all pull requests and code modifications.
```

8.2.2 2. JSON Format (`.kiro/agents/<name>.json`)

Direct JSON file containing all configuration fields, with the system prompt passed via `"prompt"`.

8.3 Field Reference & Capability Permissions

Field	Type	Description
`name`	string	Unique agent identifier (kebab-case)
`description`	string	Summary of agent role and capabilities
`model`	string	LLM model identifier (e.g., `claude-sonnet-5`)
`tools`	string[]	Built-in tools enabled (`"read"`, `"write"`, `"shell"`, etc.)
`excludedTools`	string[]	Tools explicitly disallowed
`includeMcpJson`	boolean	Inherit workspace and user MCP servers
`includePowers`	boolean	Enable dynamic power loading
`resources`	string[]	Context resources (`file://<path>`, `skill://<name>`)
`permissions.rules`	object[]	Capability rules (`capability`, `match`, `effect: allow/deny`)
`prompt`	string	System prompt (in JSON format)
`welcomeMessage`	string	Initial greeting displayed upon selecting agent

9 Bonus Lessons

9.1 Bonus Lesson 1: Kiro Web, Cloud Sessions & Cloud Configuration

- **Official Badge / ID:** ★ / `bonus-cloud`
- **Scoring Value:** 250 additional credits (requires paid plan: Pro, Pro+, Pro Max, Power)
- **Supported Surfaces:** Kiro Web, Kiro CLI, Kiro IDE
- **Official Documentation:** `https://kiro.dev/blog/agentic-engineering-in-the-cloud/`

9.1.1 Official Syllabus Definition

> "Explore agentic engineering in the cloud with Kiro Web, cloud sessions, and cloud configuration, shifting your work off your local machine. Kiro Web introduced working with Kiro in a cloud sandbox, and cloud sessions extend that to the CLI and IDE, so your Kiro sessions stay with your account and follow you across Kiro tools. Cloud configuration complements this by syncing your local Kiro setup into your cloud sessions, so your steering files, hooks, skills, powers, and custom agents can follow you as well."

9.1.2 Practical Use Cases

1. **Long-Running Refactoring:** Offload extensive multi-file migrations or framework upgrades to AWS cloud sandboxes (US East - N. Virginia) without locking local compute.
2. **Heavy Test Suites:** Run parallel property tests and comprehensive integration suites in cloud runners.
3. **Cross-Device Continuity:** Start a task on Kiro CLI or IDE locally, disconnect, and resume seamless execution in Kiro Web.
4. **Configuration Sync:** Upload local `.kiro` directories via *Settings* > *Sync* so steering files, hooks, skills, and agents persist across cloud instances.

9.2 Bonus Lesson 2: Package a Kiro Power

- **Official Badge / ID:** ★ / `bonus-2`
- **Scoring Value:** 250 additional credits (available to all plans)
- **Supported Surfaces:** Kiro IDE, Kiro CLI, Kiro Web
- **Official Documentation:** `https://kiro.dev/docs/powers/create/`

9.2.1 Official Syllabus Definition

> "In addition to our curated Kiro powers from the powers registry, developers can build their own Kiro powers to bundle relevant tools and resources for a given use case. Powers can be shared as public GitHub repositories without special approvals for publishing. Powers can be optionally submitted for review and consideration for inclusion on the curated Kiro powers registry."

9.2.2 Authoring & Packaging Workflow

1. **Create Directory:** Initialize a new repository or subfolder (e.g., `power-pulse-guard`).
2. **Manifest Definition (`plugin.json`):**

```json
{
  "$schema": "https://agent-plugins.org/schemas/1.0.0/plugin.schema.json",
  "name": "pulse-guard",
  "version": "1.0.0",
  "description": "Property verification and EARS compliance guard for Kiro",
  "author": { "name": "ArchitectOS" },
  "keywords": ["ears", "spec", "pbt", "guard", "audit"]
}
```

3. **Add Skills (`skills/<skill-name>/SKILL.md`):** Bundle actionable guidance and automation scripts.

4. **Add MCP Config (`mcp.json`):** Declare stdio or remote MCP servers required by the power.
5. **Publish:** Push to a public GitHub repository. Optionally submit for review to the curated registry via `https://kiro.dev/powers/submit`.

10 Take-Home Final Exam: Graded Build Runbook

- **Official Badge / ID:** `FIN` / `final`
- **Scoring Value:** 1,000-credit completion award (when demonstrating all 7 core lessons)
- **Submission Deadline: Monday, October 5, 2026 at 23:59 PT**
- **Entry Form:** `https://kiro.dev/2026/university/submit`

10.1 Final Submission Verification Rubric

Requirement Item	Official Rule / Constraint	Compliance Check
Repository Timing	First commit on or after **Mon Sept 21, 2026 at 09:00 PT**. No commits before. No commits after **Oct 5 at 23:59 PT** until judging concludes.	[ ] PASS
Account Age	Personal GitHub account at least **3 months old**.	[ ] PASS
Individual Entry	Solo builder (no teams). One entry per person.	[ ] PASS
`.kiro` Directory	`.kiro` folder must be present in the repository, containing files for all required lessons.	[ ] PASS
Functionality	Working project delivering real functionality (no static mockups).	[ ] PASS
Demo Video	Public video strictly between **30 seconds and 3 minutes**. (Reviewers stop watching at 3:00).	[ ] PASS
Social Post	Public post on **X** (@kirodotdev) or **LinkedIn** (@kiro) with hashtags `#KiroUniversity` and `#BuildWithKiro`.	[ ] PASS
Post Contents	Social post must include: GitHub URL, demo video, and 2–3 sentence description.	[ ] PASS
Submission Form	Completed at `https://kiro.dev/2026/university/` before Oct 5 23:59 PT with writeups for each lesson.	[ ] PASS

10.2 The Master `.kiro` Directory Blueprint

To achieve the maximum 5,250 credits, the submitted repository must organize its `.kiro` directory to clearly demonstrate all seven lessons and both bonus items:

```
my-project/
├── .kiro/
│   ├── specs/                              # [Lesson 1] Specs
│   │   └── core-engine/
│   │       ├── requirements.md             # EARS notation requirements
│   │       ├── design.md                   # Technical design & PBT invariants
│   │       └── tasks.md                    # Tasks grouped in execution waves
│   │
│   ├── steering/                           # [Lesson 2] Steering Documents
│   │   ├── tech-stack.md                   # inclusion: always
```

```
│   │   ├── api-patterns.md                 # inclusion: fileMatch
│   │   └── security-rules.md               # inclusion: auto (slash command)
│   │
│   ├── hooks/                              # [Lesson 3] Agent Hooks
│   │   ├── lint-format.json                # PostFileSave trigger + matcher
│   │   └── test-on-stop.json               # Stop trigger + confirm block
│   │
│   ├── agents/                             # [Lesson 7] Custom Agents
│   │   ├── architect.json                  # Custom tools & permissions
│   │   └── reviewer.md                     # Markdown format agent
│   │
│   └── settings/
│       └── mcp.json                        # [Lesson 6] Model Context Protocol
│
├── power-custom/                           # [Lesson 5 & Bonus 2] Packaged Power
│   ├── plugin.json                         # agent-plugins.org schema & keywords
│   ├── mcp.json                            # MCP server binding
│   └── skills/
│       └── custom-task/
│           ├── SKILL.md
│           └── scripts/
│
├── tests/
│   └── test_properties.py                  # [Lesson 4] Property-Based Tests
│                                           # (Hypothesis / fast-check invariants)
│
└── AGENTS.md                               # [Lesson 2] Root AGENTS standard file
```

---

11 Conclusion & Best Practices for Reviewers

When reviewers evaluate your project between October 5 and October 19, 2026, they verify the tangible presence and correct functioning of each artifact: 1. **Audit Traceability:** Make sure every EARS requirement in `requirements.md` maps directly to an architectural section in `design.md`, an invariant in `tests/`, and a task in `tasks.md`. 2. **Demonstrate Hooks in Video:** Show a hook firing automatically in your terminal or IDE output when the agent modifies code. 3. **Show PBT Counterexample Shrinking:** Show your property-based test running 100+ iterations and verifying universal invariants. 4. **Demonstrate Dynamic Power Activation:** Type a prompt with a power keyword and show Kiro dynamically loading the packaged tools and skills into context.

Following this runbook guarantees full compliance with all official rules, maximizes your credit score up to 5,250 credits, and establishes a rock-solid, production-grade agentic architecture.